

## PRÁCTICA GUIADA DE LABORATORIO

# MongoDB: Instalación, Base de Datos, Colecciones y Trabajo con Documentos

*NoSQL | Documentos JSON | CRUD | MongoDB Compass | mongosh*

Guía paso a paso basada en la Sesión 11: Bases de Datos NoSQL - MongoDB

| Campo             | Descripción                                                              |
|-------------------|--------------------------------------------------------------------------|
| Curso             | Base de Datos II                                                         |
| Nivel             | Universitario                                                            |
| Duración sugerida | 2 a 3 horas                                                              |
| Modalidad         | Individual o grupal                                                      |
| Herramientas      | MongoDB Community Server, MongoDB Compass y mongosh                      |
| Entregable        | Informe técnico con capturas, código ejecutado, análisis y resumen final |

*Nota metodológica: esta práctica toma como referencia los temas del material de clase: instalación de MongoDB, uso de documentos, creación de base de datos, creación de colecciones y operaciones CRUD básicas.*

## 1. Presentación de la práctica

La presente práctica tiene como finalidad que el estudiante comprenda el funcionamiento básico de MongoDB, una base de datos NoSQL orientada a documentos. A diferencia de una base de datos relacional, MongoDB almacena la información en documentos con estructura similar a JSON, lo que permite representar datos flexibles, anidados y jerárquicos.

Durante la guía, el estudiante trabajará con una base de datos de una tienda tecnológica denominada TecnoMarket Perú S.A.C., donde se administrarán clientes, productos, pedidos y proveedores. La práctica está diseñada como una guía paso a paso: primero se explica el tema, luego se desarrolla la parte práctica y finalmente se plantean actividades de análisis.

## 2. Logro de aprendizaje

Al finalizar la práctica, el estudiante crea y manipula una base de datos NoSQL en MongoDB, aplicando operaciones básicas sobre documentos y colecciones mediante comandos prácticos en mongosh o MongoDB Compass.

## 3. Competencias a desarrollar

| Competencia                   | Descripción                                                               |
|-------------------------------|---------------------------------------------------------------------------|
| Comprensión de NoSQL          | Reconoce diferencias básicas entre bases de datos relacionales y MongoDB. |
| Manejo de documentos          | Crea documentos con estructura flexible y datos anidados.                 |
| Administración de colecciones | Crea colecciones para organizar documentos.                               |
| Inserción de datos            | Aplica insertOne() e insertMany().                                        |
| Consulta de datos             | Usa find() y findOne() con condiciones y proyecciones.                    |
| Actualización de documentos   | Aplica updateOne() y updateMany() usando \$set.                           |
| Eliminación de documentos     | Usa deleteOne() y deleteMany() de manera controlada.                      |
| Análisis técnico              | Explica los resultados obtenidos en cada operación.                       |

## 4. Caso de estudio

La empresa TecnoMarket Perú S.A.C. vende productos tecnológicos como laptops, audífonos, monitores, teclados y accesorios. La empresa desea implementar una base de datos NoSQL para registrar clientes, productos, pedidos y proveedores.

El área de sistemas decide utilizar MongoDB porque necesita almacenar información flexible. Por ejemplo, algunos clientes tienen dirección completa, otros tienen varios teléfonos; algunos productos tienen características técnicas específicas, mientras que otros solo tienen datos básicos.

## 5. Parte I: Concepto básico de MongoDB

### 5.1 ¿Qué es MongoDB?

MongoDB es una base de datos NoSQL orientada a documentos. Esto significa que la información no se guarda en tablas con filas y columnas, sino en documentos similares a JSON.

```
{
  nombres: "Luis",
  apellidos: "Ramírez",
  correo: "luis.ramirez@mail.com",
  ciudad: "Lima"
}
```

## 5.2 Diferencia básica entre SQL y MongoDB

| Base de datos relacional | MongoDB                           |
|--------------------------|-----------------------------------|
| Base de datos            | Base de datos                     |
| Tabla                    | Colección                         |
| Fila o registro          | Documento                         |
| Columna                  | Campo                             |
| Clave primaria           | _id                               |
| Relación entre tablas    | Documentos anidados o referencias |

## 6. Parte II: Instalación y herramientas

### 6.1 Herramientas necesarias

| Herramienta              | Uso                                           |
|--------------------------|-----------------------------------------------|
| MongoDB Community Server | Motor de base de datos.                       |
| MongoDB Compass          | Interfaz gráfica para administrar documentos. |
| mongosh                  | Consola para ejecutar comandos.               |

### 6.2 Verificación de instalación

Abrir la terminal o consola y ejecutar el siguiente comando:

```
mongosh
```

Si MongoDB está correctamente instalado, debe aparecer una consola similar a:

```
test>
```

## 7. Parte III: Crear la base de datos

### 7.1 Explicación

En MongoDB no es necesario crear una base de datos con una instrucción especial como en SQL. Se utiliza el comando `use`. La base de datos se crea realmente cuando se inserta el primer documento.

### 7.2 Código práctico

```
use TecnoMarket
```

### 7.3 Verificar base de datos actual

```
db
```

### 7.4 Actividad para el estudiante

1. ¿Qué base de datos está usando actualmente?
2. ¿La base de datos aparece inmediatamente en la lista de bases de datos?
3. ¿Por qué MongoDB crea la base de datos recién cuando se inserta información?

## 8. Parte IV: Crear colecciones

### 8.1 Explicación

Una colección en MongoDB cumple una función similar a una tabla en una base de datos relacional. Sirve para agrupar documentos del mismo contexto.

| Colección   | Finalidad                                      |
|-------------|------------------------------------------------|
| clientes    | Guardar información de clientes.               |
| productos   | Guardar información de productos tecnológicos. |
| pedidos     | Guardar compras realizadas por clientes.       |
| proveedores | Guardar datos de empresas proveedoras.         |

### 8.2 Código práctico

```
db.createCollection("clientes")
db.createCollection("productos")
db.createCollection("pedidos")
db.createCollection("proveedores")
```

### 8.3 Verificar colecciones creadas

```
show collections
```

### 8.4 Actividad para el estudiante

1. ¿Qué colecciones se crearon?
2. ¿Qué diferencia existe entre una colección y una tabla relacional?
3. ¿MongoDB obliga a que todos los documentos de una colección tengan exactamente los mismos campos?

## 9. Parte V: Insertar documentos con insertOne()

### 9.1 Explicación

El comando insertOne() permite insertar un solo documento en una colección. Es útil cuando se registra una entidad individual, por ejemplo un cliente nuevo.

### 9.2 Insertar un cliente

```
db.clientes.insertOne({
  nombres: "Luis",
  apellidos: "Ramírez",
  dni: "12345678",
  correo: "luis.ramirez@mail.com",
  telefono: "987654321",
  direccion: {
    ciudad: "Lima",
    distrito: "Los Olivos",
    calle: "Av. Universitaria 123"
  },
  fechaRegistro: new Date(),
  estado: "Activo"
})
```

### 9.3 Explicación del documento

| Campo         | Descripción                                     |
|---------------|-------------------------------------------------|
| nombres       | Nombre del cliente.                             |
| apellidos     | Apellidos del cliente.                          |
| dni           | Documento de identidad.                         |
| correo        | Correo electrónico.                             |
| telefono      | Número de contacto.                             |
| direccion     | Documento anidado con ciudad, distrito y calle. |
| fechaRegistro | Fecha automática de registro.                   |
| estado        | Situación del cliente.                          |

### 9.4 Verificar inserción

```
db.clientes.find()
```

### 9.5 Actividad para el estudiante

Inserte un nuevo cliente con datos de prueba. Debe incluir nombres, apellidos, DNI, correo, teléfono, dirección anidada y estado.

## 10. Parte VI: Insertar varios documentos con insertMany()

### 10.1 Explicación

El comando insertMany() permite insertar varios documentos al mismo tiempo. Es útil cuando se desea cargar datos iniciales en una colección.

### 10.2 Insertar productos

```
db.productos.insertMany([
  {
    codigo: "P001",
    nombre: "Laptop Lenovo IdeaPad",
    categoria: "Laptops",
    marca: "Lenovo",
    precio: 2800,
    stock: 8,
    caracteristicas: {
      procesador: "Intel Core i5",
      ram: "16GB",
      almacenamiento: "512GB SSD"
    },
    estado: "Disponible"
  },
  {
    codigo: "P002",
    nombre: "Mouse Logitech M170",
    categoria: "Accesorios",
    marca: "Logitech",
    precio: 65,
    stock: 30,
    caracteristicas: {
      tipo: "Inalámbrico",
      color: "Negro"
    }
  }
])
```

```

    },
    estado: "Disponible"
  },
  {
    codigo: "P003",
    nombre: "Monitor Samsung 24 pulgadas",
    categoria: "Monitores",
    marca: "Samsung",
    precio: 620,
    stock: 5,
    caracteristicas: {
      tamano: "24 pulgadas",
      resolucion: "Full HD"
    },
    estado: "Disponible"
  },
  {
    codigo: "P004",
    nombre: "Teclado Mecánico Redragon",
    categoria: "Accesorios",
    marca: "Redragon",
    precio: 180,
    stock: 12,
    caracteristicas: {
      tipo: "Mecánico",
      iluminacion: "RGB"
    },
    estado: "Disponible"
  }
]
})

```

### 10.3 Verificar productos

```

db.productos.find()
db.productos.find().pretty()

```

### 10.4 Actividad para el estudiante

Agregue tres productos nuevos relacionados con tecnología. Cada producto debe tener código, nombre, categoría, marca, precio, stock, características y estado.

## 11. Parte VII: Consultar documentos con find()

### 11.1 Explicación

El comando find() permite consultar documentos de una colección. Puede mostrar todos los documentos o aplicar condiciones.

### 11.2 Consultas prácticas

```

// Consultar todos los productos
db.productos.find()

// Consultar productos con stock mayor a 10
db.productos.find({ stock: { $gt: 10 } })

// Consultar productos con precio menor a 500
db.productos.find({ precio: { $lt: 500 } })

```

```
// Consultar productos por categoría
db.productos.find({ categoria: "Accesorios" })

// Consultar productos disponibles
db.productos.find({ estado: "Disponible" })
```

### 11.3 Actividad para el estudiante

1. ¿Cuántos productos tienen stock mayor a 10?
2. ¿Qué productos cuestan menos de S/ 500?
3. ¿Qué productos pertenecen a la categoría Accesorios?
4. ¿Qué utilidad tiene consultar documentos con condiciones?

## 12. Parte VIII: Consultar un documento con findOne()

### 12.1 Explicación

El comando findOne() devuelve un solo documento que cumple con una condición. Es útil cuando se busca un dato específico.

### 12.2 Código práctico

```
// Buscar un cliente por DNI
db.clientes.findOne({ dni: "12345678" })

// Buscar un producto por código
db.productos.findOne({ codigo: "P001" })
```

### 12.3 Actividad para el estudiante

1. Realice una consulta para buscar un producto por marca.
2. ¿Qué diferencia observa entre find() y findOne()?
3. ¿Cuándo conviene usar findOne()?

## 13. Parte IX: Consultar campos específicos: proyección

### 13.1 Explicación

MongoDB permite mostrar solo algunos campos del documento. Esto se denomina proyección. La proyección ayuda a reducir la cantidad de información mostrada cuando solo se necesitan ciertos datos.

### 13.2 Código práctico

```
// Mostrar solo nombre, precio y stock
db.productos.find(
  {},
  { _id: 0, nombre: 1, precio: 1, stock: 1 }
)

// Mostrar clientes sin el campo _id
db.clientes.find(
  {},
  { _id: 0, nombres: 1, apellidos: 1, correo: 1 }
)
```

### 13.3 Actividad para el estudiante

Cree una consulta que muestre solamente el código del producto, nombre, categoría y precio.

## 14. Parte X: Insertar pedidos con documentos anidados

### 14.1 Explicación

Una ventaja importante de MongoDB es que permite almacenar información relacionada dentro del mismo documento. Por ejemplo, un pedido puede tener los datos del cliente y una lista de productos comprados.

### 14.2 Insertar pedidos

```
db.pedidos.insertMany([
  {
    numeroPedido: "PED001",
    cliente: {
      dni: "12345678",
      nombres: "Luis Ramírez",
      ciudad: "Lima"
    },
    productos: [
      { codigo: "P001", nombre: "Laptop Lenovo IdeaPad", cantidad: 1, precioUnitario: 2800 },
      { codigo: "P002", nombre: "Mouse Logitech M170", cantidad: 2, precioUnitario: 65 }
    ],
    total: 2930,
    estado: "Pagado",
    fechaPedido: new Date("2024-07-10")
  },
  {
    numeroPedido: "PED002",
    cliente: {
      dni: "23456789",
      nombres: "María Torres",
      ciudad: "Arequipa"
    },
    productos: [
      { codigo: "P003", nombre: "Monitor Samsung 24 pulgadas", cantidad: 1, precioUnitario: 620 }
    ],
    total: 620,
    estado: "Pendiente",
    fechaPedido: new Date("2024-07-12")
  }
])
```

### 14.3 Consultar pedidos

```
db.pedidos.find().pretty()
```

### 14.4 Actividad para el estudiante

Inserte un nuevo pedido con número de pedido, cliente, mínimo dos productos, total, estado y fecha del pedido.

## 15. Parte XI: Consultas sobre documentos anidados

### 15.1 Explicación

MongoDB permite consultar campos internos de un documento usando notación de punto. Por ejemplo, para consultar la ciudad del cliente dentro de un pedido se usa "cliente.ciudad".

## 15.2 Código práctico

```
// Buscar pedidos de clientes de Lima
db.pedidos.find({ "cliente.ciudad": "Lima" })

// Buscar pedidos que tengan un producto específico
db.pedidos.find({ "productos.codigo": "P001" })

// Buscar pedidos pagados
db.pedidos.find({ estado: "Pagado" })

// Buscar pedidos con total mayor a 1000
db.pedidos.find({ total: { $gt: 1000 } })
```

## 15.3 Actividad para el estudiante

1. ¿Qué pedidos pertenecen a clientes de Lima?
2. ¿Qué pedidos incluyen el producto P001?
3. ¿Qué pedidos tienen total mayor a S/ 1000?
4. ¿Por qué MongoDB permite consultar campos internos usando punto, como "cliente.ciudad"?

## 16. Parte XII: Actualizar documentos con updateOne()

### 16.1 Explicación

El comando `updateOne()` actualiza un solo documento que cumple una condición. Para modificar campos se utiliza el operador `$set`.

### 16.2 Código práctico

```
// Actualizar precio de un producto
db.productos.updateOne(
  { codigo: "P002" },
  { $set: { precio: 70 } }
)

// Verificar actualización
db.productos.findOne({ codigo: "P002" })

// Actualizar estado de un pedido
db.pedidos.updateOne(
  { numeroPedido: "PED002" },
  { $set: { estado: "Pagado" } }
)
```

### 16.3 Actividad para el estudiante

Actualice el stock de un producto específico y luego verifique el resultado con `findOne()`.

## 17. Parte XIII: Actualizar varios documentos con updateMany()

### 17.1 Explicación

El comando `updateMany()` actualiza varios documentos que cumplen una condición. Es útil cuando se desea aplicar un cambio masivo controlado.

## 17.2 Código práctico

```
// Cambiar estado de productos con poco stock
db.productos.updateMany(
  { stock: { $lte: 5 } },
  { $set: { estado: "Stock bajo" } }
)

// Verificar productos con stock bajo
db.productos.find({ estado: "Stock bajo" })

// Agregar promoción a accesorios
db.productos.updateMany(
  { categoria: "Accesorios" },
  { $set: { enPromocion: true } }
)
```

## 17.3 Actividad para el estudiante

Actualice todos los productos de una categoría agregando el campo enPromocion: true. Luego consulte los documentos modificados.

## 18. Parte XIV: Uso de operadores adicionales

### 18.1 Operadores básicos

| Operador | Significado         | Ejemplo                                           |
|----------|---------------------|---------------------------------------------------|
| \$gt     | Mayor que           | { precio: { \$gt: 500 } }                         |
| \$lt     | Menor que           | { precio: { \$lt: 500 } }                         |
| \$gte    | Mayor o igual que   | { stock: { \$gte: 10 } }                          |
| \$lte    | Menor o igual que   | { stock: { \$lte: 5 } }                           |
| \$ne     | Diferente de        | { estado: { \$ne: "Disponible" } }                |
| \$in     | Dentro de una lista | { categoria: { \$in: ["Laptops", "Monitores"] } } |

### 18.2 Código práctico

```
// Productos entre S/ 100 y S/ 1000
db.productos.find({ precio: { $gte: 100, $lte: 1000 } })

// Productos de categorías específicas
db.productos.find({ categoria: { $in: ["Laptops", "Monitores"] } })

// Productos que no están disponibles
db.productos.find({ estado: { $ne: "Disponible" } })
```

### 18.3 Actividad para el estudiante

1. Cree una consulta usando \$gte.
2. Cree una consulta usando \$lte.
3. Cree una consulta usando \$in.

## 19. Parte XV: Eliminar documentos con deleteOne()

### 19.1 Explicación

El comando deleteOne() elimina un solo documento que cumple una condición. Debe utilizarse con cuidado para evitar eliminar información incorrecta.

### 19.2 Código práctico

```
// Insertar proveedor de prueba
db.proveedores.insertOne({
  ruc: "20123456789",
  razonSocial: "Importaciones Tecnológicas Perú S.A.C.",
  contacto: "Carlos Salazar",
  telefono: "999888777",
  ciudad: "Lima",
  estado: "Activo"
})

// Eliminar proveedor por RUC
db.proveedores.deleteOne({ ruc: "20123456789" })

// Verificar eliminación
db.proveedores.find()
```

### 19.3 Actividad para el estudiante

Inserte dos proveedores y elimine solo uno utilizando una condición específica.

## 20. Parte XVI: Eliminar varios documentos con deleteMany()

### 20.1 Explicación

El comando deleteMany() elimina todos los documentos que cumplen una condición. Se debe usar con mayor cuidado que deleteOne(), porque puede eliminar varios registros en una sola operación.

### 20.2 Código práctico

```
db.proveedores.insertMany([
  { ruc: "20444444441", razonSocial: "Accesorios Globales S.A.C.", ciudad: "Lima", estado: "Inactivo" },
  { ruc: "20444444442", razonSocial: "Tecnología Mayorista Perú S.R.L.", ciudad: "Arequipa", estado: "Activo" },
  { ruc: "20444444443", razonSocial: "Distribuidora Digital Norte S.A.C.", ciudad: "Trujillo", estado: "Inactivo" }
])

// Eliminar proveedores inactivos
db.proveedores.deleteMany({ estado: "Inactivo" })

// Verificar eliminación
db.proveedores.find()
```

### 20.3 Actividad para el estudiante

Explique por qué deleteMany() debe usarse con condiciones bien definidas.

## 21. Parte XVII: Consultas finales de repaso

Ejecute las siguientes consultas y capture los resultados:

```
// 1. Listar todos los clientes
db.clientes.find().pretty()

// 2. Listar productos disponibles
db.productos.find({ estado: "Disponible" }).pretty()

// 3. Listar productos con precio mayor a S/ 500
db.productos.find({ precio: { $gt: 500 } }).pretty()

// 4. Listar pedidos pagados
db.pedidos.find({ estado: "Pagado" }).pretty()

// 5. Listar pedidos de clientes de Lima
db.pedidos.find({ "cliente.ciudad": "Lima" }).pretty()

// 6. Mostrar solo nombre y precio de productos
db.productos.find({}, { _id: 0, nombre: 1, precio: 1 })
```

## 22. Actividad de análisis

1. ¿Por qué MongoDB es considerado una base de datos NoSQL?
2. ¿Qué ventaja tiene almacenar datos en documentos?
3. ¿Qué diferencia existe entre una colección y una tabla?
4. ¿Qué diferencia existe entre un documento y una fila?
5. ¿Qué ventaja tiene usar documentos anidados?
6. ¿Qué operación permite insertar un solo documento?
7. ¿Qué operación permite insertar varios documentos?
8. ¿Qué diferencia existe entre find() y findOne()?
9. ¿Qué función cumple \$set en una actualización?
10. ¿Qué riesgo existe al usar deleteMany() sin una condición adecuada?
11. ¿En qué tipo de aplicaciones sería útil usar MongoDB?
12. ¿Qué aprendió al trabajar con documentos JSON?

## 23. Entregable final

| Sección         | Contenido esperado                                                            |
|-----------------|-------------------------------------------------------------------------------|
| Carátula        | Nombre del estudiante, curso, docente, fecha y título de la práctica.         |
| Introducción    | Explicación breve sobre MongoDB y bases de datos NoSQL.                       |
| Desarrollo      | Capturas de creación de base de datos, colecciones e inserción de documentos. |
| Consultas       | Evidencias de consultas con find(), findOne() y operadores.                   |
| Actualizaciones | Evidencias de updateOne() y updateMany().                                     |
| Eliminaciones   | Evidencias de deleteOne() y deleteMany().                                     |
| Análisis        | Respuestas a las preguntas planteadas.                                        |
| Resumen final   | Explicación de lo aprendido en la guía.                                       |
| Anexos          | Código completo ejecutado en MongoDB.                                         |

## 24. Rúbrica de evaluación

| Criterio                                | Excelente - 4 pts                                                                     | Bueno - 3 pts                                              | Regular - 2 pts                                     | Deficiente - 1 pt                               |
|-----------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------|
| Creación de base de datos y colecciones | Crea correctamente la base de datos y todas las colecciones solicitadas.              | Crea la mayoría de elementos con errores menores.          | Crea parcialmente las colecciones.                  | No logra crear la base de datos ni colecciones. |
| Inserción de documentos                 | Inserta documentos completos, coherentes y con estructuras anidadas.                  | Inserta documentos adecuados con pocos detalles faltantes. | Inserta pocos documentos o con estructura limitada. | No inserta documentos correctamente.            |
| Consultas en MongoDB                    | Aplica correctamente find(), findOne(), condiciones y proyecciones.                   | Realiza la mayoría de consultas correctamente.             | Presenta consultas básicas con errores.             | No evidencia consultas funcionales.             |
| Actualización y eliminación             | Usa correctamente updateOne(), updateMany(), deleteOne() y deleteMany().              | Aplica operaciones con pequeños errores.                   | Aplica solo algunas operaciones.                    | No logra actualizar ni eliminar documentos.     |
| Análisis e interpretación               | Explica con claridad el funcionamiento de documentos, colecciones y operaciones CRUD. | Presenta análisis general aceptable.                       | Análisis superficial.                               | No interpreta los resultados.                   |
| Presentación del informe                | Presenta evidencias, código y conclusiones de forma ordenada y profesional.           | Presenta informe claro con mínimos errores.                | Informe incompleto o desordenado.                   | No cumple la estructura solicitada.             |

Puntaje sugerido: 24 puntos. Para convertir a escala vigesimal, dividir el puntaje obtenido entre 24 y multiplicar por 20.

## 25. Resumen de todo lo realizado en la guía

En esta práctica se trabajó con MongoDB como base de datos NoSQL orientada a documentos. Primero se revisó la importancia de MongoDB para manejar información flexible, escalable y semiestructurada. Luego se explicó la diferencia entre una base de datos relacional y MongoDB, identificando equivalencias como tabla/colección, fila/documento y columna/campo.

Después se creó la base de datos TecnoMarket y las colecciones clientes, productos, pedidos y proveedores. Se insertaron documentos individuales con insertOne() y múltiples documentos con insertMany(). Además, se trabajó con documentos anidados, especialmente en direcciones, características de productos y pedidos con listas de productos.

Posteriormente se realizaron consultas con find() y findOne(), aplicando condiciones mediante operadores como \$gt, \$lt, \$gte, \$lte, \$ne y \$in. También se aplicó proyección para mostrar solo algunos campos de los documentos.

Finalmente, se actualizaron documentos con updateOne() y updateMany() usando \$set, y se eliminaron documentos mediante deleteOne() y deleteMany(). Con ello, el estudiante desarrolló una visión práctica del ciclo CRUD en MongoDB: crear, consultar, actualizar y eliminar documentos en una base de datos NoSQL.

## 26. Anexo A: Script completo para ejecutar en mongosh

```
use TecnoMarket

db.createCollection("clientes")
db.createCollection("productos")
db.createCollection("pedidos")
db.createCollection("proveedores")

db.clientes.insertOne({
```

```

nombres: "Luis",
apellidos: "Ramírez",
dni: "12345678",
correo: "luis.ramirez@mail.com",
telefono: "987654321",
direccion: { ciudad: "Lima", distrito: "Los Olivos", calle: "Av. Universitaria 123" },
fechaRegistro: new Date(),
estado: "Activo"
})

db.productos.insertMany([
  { codigo: "P001", nombre: "Laptop Lenovo IdeaPad", categoria: "Laptops", marca: "Lenovo", precio: 2800, stock: 8,
    características: { procesador: "Intel Core i5", ram: "16GB", almacenamiento: "512GB SSD" }, estado: "Disponible" },
  { codigo: "P002", nombre: "Mouse Logitech M170", categoria: "Accesorios", marca: "Logitech", precio: 65, stock: 30,
    características: { tipo: "Inalámbrico", color: "Negro" }, estado: "Disponible" },
  { codigo: "P003", nombre: "Monitor Samsung 24 pulgadas", categoria: "Monitores", marca: "Samsung", precio: 620, stock: 5,
    características: { tamaño: "24 pulgadas", resolución: "Full HD" }, estado: "Disponible" },
  { codigo: "P004", nombre: "Teclado Mecánico Redragon", categoria: "Accesorios", marca: "Redragon", precio: 180, stock: 12,
    características: { tipo: "Mecánico", iluminación: "RGB" }, estado: "Disponible" }
])

db.pedidos.insertMany([
  { numeroPedido: "PED001", cliente: { dni: "12345678", nombres: "Luis Ramírez", ciudad: "Lima" }, productos: [
    { codigo: "P001", nombre: "Laptop Lenovo IdeaPad", cantidad: 1, precioUnitario: 2800 }, { codigo: "P002", nombre: "Mouse Logitech M170", cantidad: 2, precioUnitario: 65 } ], total: 2930, estado: "Pagado", fechaPedido: new Date("2024-07-10") },
  { numeroPedido: "PED002", cliente: { dni: "23456789", nombres: "María Torres", ciudad: "Arequipa" }, productos: [
    { codigo: "P003", nombre: "Monitor Samsung 24 pulgadas", cantidad: 1, precioUnitario: 620 } ], total: 620, estado: "Pendiente", fechaPedido: new Date("2024-07-12") }
])

db.productos.find().pretty()
db.productos.find({ stock: { $gt: 10 } })
db.productos.find({ precio: { $lt: 500 } })
db.productos.find({ categoria: "Accesorios" })
db.clientes.findOne({ dni: "12345678" })
db.productos.findOne({ codigo: "P001" })
db.productos.find({}, { _id: 0, nombre: 1, precio: 1, stock: 1 })
db.pedidos.find({ "cliente.ciudad": "Lima" })
db.pedidos.find({ "productos.codigo": "P001" })
db.pedidos.find({ total: { $gt: 1000 } })

db.productos.updateOne({ codigo: "P002" }, { $set: { precio: 70 } })
db.pedidos.updateOne({ numeroPedido: "PED002" }, { $set: { estado: "Pagado" } })
db.productos.updateMany({ stock: { $lte: 5 } }, { $set: { estado: "Stock bajo" } })
db.productos.updateMany({ categoria: "Accesorios" }, { $set: { enPromocion: true } })

db.proveedores.insertMany([
  { ruc: "20444444441", razonSocial: "Accesorios Globales S.A.C.", ciudad: "Lima", estado: "Inactivo" },
  { ruc: "20444444442", razonSocial: "Tecnología Mayorista Perú S.R.L.", ciudad: "Arequipa", estado: "Activo" },
  { ruc: "20444444443", razonSocial: "Distribuidora Digital Norte S.A.C.", ciudad: "Trujillo", estado: "Inactivo" }
])

db.proveedores.deleteOne({ ruc: "20444444441" })
db.proveedores.deleteMany({ estado: "Inactivo" })

```